

EXPERIMENT - 8

Title

Perform Count, Limit, Sort, and Skip Operations on the Given Collections using MongoDB

Aim

To implement and execute MongoDB operations such as:

- **Count**
 - **Limit**
 - **Sort**
 - **Skip**
on a given collection.
-

Prerequisites

- Basic knowledge of MongoDB
 - Completion of Experiment 6 and 7
 - MongoDB installed and running
-

Software Requirements

- Operating System: Windows / Linux / macOS
 - Database: MongoDB Community Server
 - Tool: MongoDB Shell (mongosh)
-

Hardware Requirements

- Processor: Intel i3 or above
 - RAM: Minimum 4 GB
-

Theory

MongoDB provides advanced query operations to control how data is retrieved:

1) Count

Used to count the number of documents in a collection.

2) Limit

Restricts the number of documents returned.

3) Sort

Sorts documents in ascending or descending order.

4) Skip

Skips a specified number of documents (used for pagination).

These operations are useful in:

- Pagination
 - Ranking systems
 - Reporting
 - Data analysis
-

Dataset Used

Collection Name: restaurants

Sample Dataset:

```
db.restaurants.insertMany([
  { name: "Spice Hub", location: "Hyderabad", rating: 4.5 },
```

```
{ name: "Food Palace", location: "Secunderabad", rating: 4.2 },  
{ name: "Green Leaf", location: "Gachibowli", rating: 4.7 },  
{ name: "Tasty Treat", location: "Madhapur", rating: 4.1 },  
{ name: "Urban Bites", location: "Kukatpally", rating: 4.8 }  
])
```

Procedure & Commands

→ Step 1: Use Database

```
use foodDeliveryDB
```

→ Step 2: Count Operation

Count All Documents

```
db.restaurants.countDocuments()
```

Count with Condition

```
db.restaurants.countDocuments({ location: "Hyderabad" })
```

→ Step 3: Limit Operation

Display First 2 Records

```
db.restaurants.find().limit(2)
```

→ Step 4: Sort Operation

Sort by Rating (Ascending)

```
db.restaurants.find().sort({ rating: 1 })
```

Sort by Rating (Descending)

```
db.restaurants.find().sort({ rating: -1 })
```

→ Step 5: Skip Operation

Skip First 2 Records

```
db.restaurants.find().skip(2)
```

→ Step 6: Combined Operations (Pagination Example)

```
db.restaurants.find()  
  .sort({ rating: -1 })  
  .skip(1)  
  .limit(2)
```

This:

- Sorts by highest rating
 - Skips first record
 - Displays next 2 records
-

Sample Output

```
5  (Total documents count)  
{ name: "Urban Bites", rating: 4.8 }  
{ name: "Green Leaf", rating: 4.7 }
```

Expected Result

MongoDB successfully performs:

- Document counting
 - Limiting result set
 - Sorting documents
 - Skipping records
on the given collection.
-

Result

Thus, the Count, Limit, Sort, and Skip operations were successfully implemented and executed using MongoDB.

Viva Voce Questions

1. What is the purpose of `countDocuments()`?
 2. What is the difference between `sort(1)` and `sort(-1)`?
 3. How is skip used in pagination?
 4. Can limit and sort be combined?
 5. Which operation is executed first in chained queries?
 6. Why are these operations important in real applications?
-

Conclusion

This experiment demonstrates advanced MongoDB querying techniques essential for handling large datasets and implementing pagination in full stack applications.